TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Introduction……. 3

1.2 Problem Statement …….…… 4

1.3 Objectives ……...….... 4

1.4 Methodology ……... 5-7

CHAPTER 2: LITERATURE SURVEY

2.1 Literature Survey …….…….. 11-12

CHAPTER 3: DATASET

3.1 Dataset.…….……..……..... 13-14

CHAPTER 4: SYSTEM DESIGN

4.1 System Design.……..……..... 16-19

CHAPTER 5: IMPLEMENTATION

5.1 Implementation …….……..... 21-22

5.2 Code …….…….……. 23-28

CHAPTER 6: RESULT AND DISCUSSIONS

6.1 Results and Discussion ……...…… 30

6.2 Output Image …….. 31

CHAPTER 7: CONCLUSION

7.1 Conclusion ……..…… 33

CHAPTER 8: REFERNCES

8.1 References ……...…….…..…. 35

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Fire accidents pose a serious threat to industries, crowded events, social gatherings, and densely populated areas that are observed across India. These kinds of incidents may cause damage to property, environment, and pose a threat to human and animal life. According to the recent National Risk Survey Report, Fire stood at the third position overtaking corruption, terrorism, and insurgency thus posing a significant risk to our country's economy and citizens.

Early detection of fire-accidents can save innumerable lives along with saving properties from permanent infrastructure damage and the consequent financial losses. In order to achieve high accuracy and robustness in dense urban areas, detection through local surveillance is necessary and also effective. Traditional opto-electronic fire detection systems have major disadvantages: Requirement of separate and often redundant systems, fault-prone hardware systems, regular maintenance, false alarms and so on. Usage of sensors in hot, dusty industrial conditions is also not possible. Thus, detecting fires through surveillance video stream is one of the most feasible, cost-effective solution suitable for replacement of existing systems without the need for large infrastructure installation or investment. The existing video-based machine learning models rely heavily on domain knowledge and feature engineering to achieve detection therefore, have to be updated to meet new threats.

We aim to develop a classification model using Deep learning and Transfer Learning to recognise fires in images/video frames, thus ensuring early detection and save manual work. This model can be used to detect fires in surveillance videos. Unlike existing systems, this neither requires special infrastructure for setup like hardware-based solutions, nor does it need domain knowledge and prohibitive computation for development.

1.2 PROBLEM STATEMENT

With the increasing frequency of fire-related incidents worldwide, the need for early detection and response to fire hazards has become paramount. Traditional fire detection systems, which rely on smoke detectors and temperature sensors, often provide alerts after a significant delay or can trigger false alarms. This delay can result in devastating property damage and loss of life. Traditional opto-electronic fire detection systems have major disadvantages: Requirement of separate and often redundant systems, fault-prone hardware systems, regular maintenance, false alarms and so on. Usage of sensors in hot, dusty industrial conditions is also not possible., enhancing usability and reducing the reliance on manual intervention.

1.3 OBJECTIVES

Developing a robust deep learning model capable of accurately identifying fire in real-time from live video feeds or images, with high precision and minimal false positives.

• **Real-time Processing:** Ensure the system processes data in real-time to provide immediate detection of fire hazards, reducing response time and potentially saving lives and property.

• **Early Warning System:** Integrate the detection system with an alarm system that triggers instant alerts as soon as fire is detected, ensuring prompt evacuation or intervention.

• **Minimization of False Alarms:** Implement a system that can effectively differentiate between actual fire events and other similar visual disturbances (such as sunlight, reflections, etc.), minimizing the occurrence of false alarms.

1.4 METHODOLGY

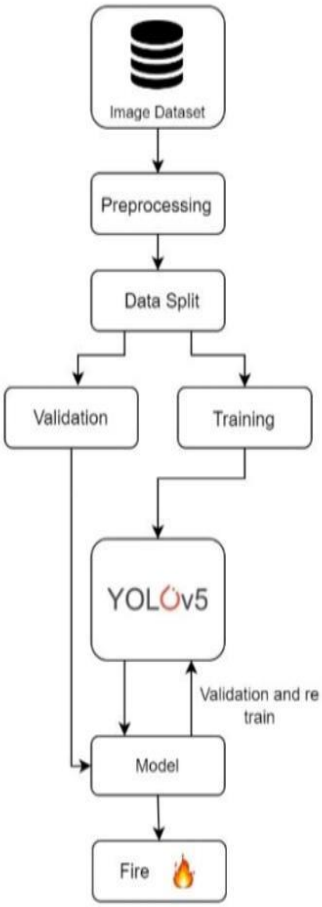


Fig 1.1: METHODOLGY

The methodology of fire detection with alarm system involves a series of steps and techniques. The passive components of the system include data preprocessing, features engineering, model selection scripts which were used to train and develop machine learning model. The detailed overview of how each step was approached to create effective and coherent video content.

Data Collection and Preparation:

The pretrained weights are especially beneficial because they:

- Reduce Training Time: Starting with a pretrained model eliminates the need to train from scratch, significantly reducing computational resources and time.
- Improve Accuracy: Pretrained weights offer better initial performance since the model has already learned basic visual features like edges, textures, and shapes from a vast dataset.
- Facilitate Transfer Learning: The model is fine-tuned on our specific fire and smoke detection dataset, allowing it to specialize in identifying fire and smoke patterns effectively.

Model Selection:

In this project, YOLOv5 was selected as the core model for fire and smoke detection due to its superior balance between speed, accuracy, and ease of use. YOLOv5, an advanced version of the "You Only Look Once" family of models, is a state-of-the-art object detection framework known for its robust real-time detection capabilities.

YOLOv5 was trained and fine-tuned on a custom dataset annotated for fire detection. Pretrained weights from Roboflow were used as a starting point to accelerate the training process. The model achieved high accuracy and was deployed for real-time monitoring through video streams.

Model Training and Evaluation:

In this project, the model training and evaluation were conducted using pretrained weights from Roboflow. These pretrained weights provided a solid foundation, enabling the YOLOv5 model to start with prior knowledge from a wide range of general object detection tasks. This approach significantly reduced the training time and improved the model's ability to learn relevant features quickly. By fine-tuning the model on the custom fire and smoke dataset, we were able to adapt the pretrained YOLOv5 model to the specific task of fire and smoke detection.

The training process involved uploading the custom dataset to Roboflow, where it was pre-processed and augmented using techniques like flipping, rotation,

and brightness adjustments to simulate real-world variations. The dataset was then split into training, validation, and test sets, and the model was trained using these sets. The model's performance was evaluated using metrics like precision, recall, F1-score, and mean average precision (mAP) to ensure it was accurately identifying fire and smoke in different conditions. Using pretrained weights from Roboflow not only accelerated the training process but also contributed to higher detection accuracy. The fine-tuned YOLOv5 model demonstrated strong performance in detecting fire and smoke, showing robustness in diverse environments, including those with challenging lighting conditions or partial obstructions. Overall, this approach helped streamline the training and evaluation phases, leading to an effective fire and smoke detection system.

Deployment:

After successfully training and evaluating the YOLOv5 model using Roboflow pretrained weights, the next crucial step is deploying the model for real-time fire and smoke detection. The deployment phase focuses on integrating the trained model into a production environment where it can continuously monitor video feeds and trigger alerts when fire is detected.

Real-Time Video Stream Integration:

For real-time monitoring, the system is integrated with video surveillance streams, such as CCTV cameras or web cams. The model processes each frame of the video in real time, detecting fire and smoke and drawing bounding boxes around the affected areas. This is done using frameworks like OpenCV, which captures video frames and passes them to the trained model for inference.

The YOLOv5 model, once deployed, performs inference on these frames, detecting fire regions. The bounding boxes generated by the model, along with

confidence scores, are overlaid on the video in real time. This allows operators to visually monitor potential hazards as they are identified.

Alert and Notification System:

The deployment includes an alert mechanism that triggers when fire is detected in the video feed. Once the model identifies a potential threat, it sends notifications through email and triggers the alarm.

CHAPTER 2
LITERATURE SURVEY

2.1 LITERATURE SURVEY

The literature survey gives a comprehensive review of existing research and studies on generative video models, summarizing key findings, methodologies, and gaps. This helps us to contextualize the current work and identify areas for further research.

The method is based on a pipeline model that includes Background Subtraction, Color Segmentation, Spatial Wavelet Analysis, and Support Vector Machine (SVM) classification. The system detects fire by analyzing video feeds using a combination of motion detection, color segmentation, and wavelet analysis, with classification by SVM. [1].

The proposed system uses Convolutional Neural Networks (CNN) for fire detection, YOLO (You Only Look Once) architecture for object detection, and a depth estimation technique for fire localization and spread rate estimation. The system processes surveillance videos frame by frame and analyzes whether fire is present. If fire is detected, the system provides additional information such as fire location, contact objects, and the spread rate of the fire [2].

The study uses YOLOv5 for fire detection. YOLO (You Only Look Once) is a one-stage object detection algorithm that divides the image into grids and detects objects within each grid. YOLOv5 uses the PyTorch framework and YOLOv5s (a smaller, faster version) for training to enhance real-time detection capabilities. The process involves pre-training a model on labeled fire images and then deploying the model for real-time fire detection in edge devices with limited computational resources. [3].

The system uses Convolutional Neural Networks (CNN), particularly the YOLOv3 model, for forest fire detection. The approach involves preprocessing (image annotation, de-slanting), feature extraction using CSP (Cross Space Partial) and Darknet53 algorithms, and classification with YOLOv3 to identify fire outbreaks. SegYOLO and Grabcut algorithms were used for segmentation [4].

A deep CNN is used to classify whether a given frame contains a fire or not. Convolutional layers allow the model to learn spatial hierarchies of features, making it suitable for detecting complex visual patterns like flames, smoke, and fire. [5].

This operates by combining various features extracted from video frame such as color, motion, texture, and temporal dynamic to detect fire accurately and efficiently. Each video frame is analysed to extract multiple fire-relevant features. These features act as complementary inputs for the detection model. [6].

Author(s)	Title	Methodology	Dataset	Limitations
[1] Md. Abdur Rahman, Sayed Tanimun Hasan, Mohammed Abdul Kader	Computer Vision Based Industrial and Forest Fire Detection Using Support Vector Machine (SVM)	Support Vector Machine (SVM) with Radial Basis Function (RBF) kernel	500 wavelet energies from real fire videos and 500 non-fire videos.	It includes memory and processing power requirements and false positives.
[2] Peter Janků, Zuzana Komínková Oplatková, Tomáš Dulík, Petr Snopek, Jiří Líba	Fire Detection and Analysis in Video Stream by Using Simple Artificial Neural Network	Artificial Neural Network (ANN) Model	2725 images	False positives due to objects with fire-like colors.
[3] Wanbo Luo	Research on Fire Detection Based on YOLOv5	YOLOv5 for fire detection.	2065 images	The model's performance should be improved in more complex environments.
[4] Mr. M. Karthi, Dr. R. Priscilla, Ms. Subhashini G., Ms. Niroshini Infantia C., Mr. Abijith G. R., and Ms. Vinisha J.	Forest Fire Detection: A Comparative Analysis of Deep Learning Algorithms	Convolutional Neural Networks (CNN), YOLOv3 model	7000 images	False positives and optimizing the system for different environments remains.
[5] K. Muhammad, J. Ahmad, Z. Lv, P. Bellavista, P. Yang, and S. W. Baik.	Efficient Deep CNN-Based Fire Detection and Localization in Video Surveillance Applications	Deep Convolutional Neural Network Model	62690 Video Frames	The detection and localization of fire may be slower or less accurate. Lesser accurate detection.
[6] Faming Gong, Chuantao Li, Wenjuan Gong, Xin Li, Xiangbing Yuan, Yuhui Ma, Tao Song.	A Real-Time Fire Detection Method from Video with Multi-feature Fusion	Fusion model	66690 video frames.	The feature extraction process is han-engineered which limits generalization to new, unseen environment. False detection for similar color and motion patterns.

CHAPTER 3

DATASETS

3.1 DATASETS

According to the objective of the experiments, the dataset is divided into three parts: training dataset (train), verification dataset (val), and test dataset (test). The training dataset is used to train the model, whereas the verification dataset is used to evaluate the model's performance. The test dataset is used to evaluate the model's efficiency, precision, and generalizability. The training dataset (consisting of 1721 images) used in this article is collected from experiments, public datasets, and Internet photographs and videos of fire. All photos are randomly divided into a training dataset and a verification dataset in the ratio of 7:3, with 1210 images automatically assigned to the training dataset and 510 images assigned to the verification dataset. The employed dataset consists of fire photos with various shapes, distances, and interfering objects from various combustion objects (buildings, automobiles) and scenarios. The photographs depict both indoor and outdoor themes, such as families, offices, and factories, as well as mountains, forests, and roads.

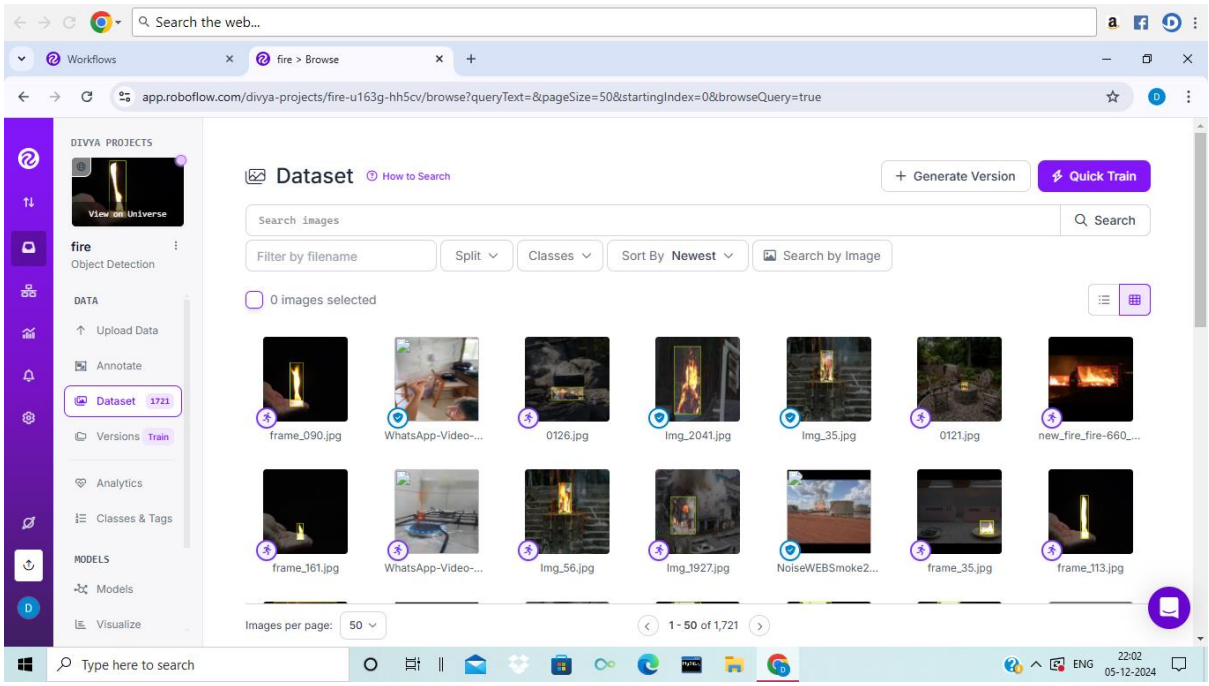
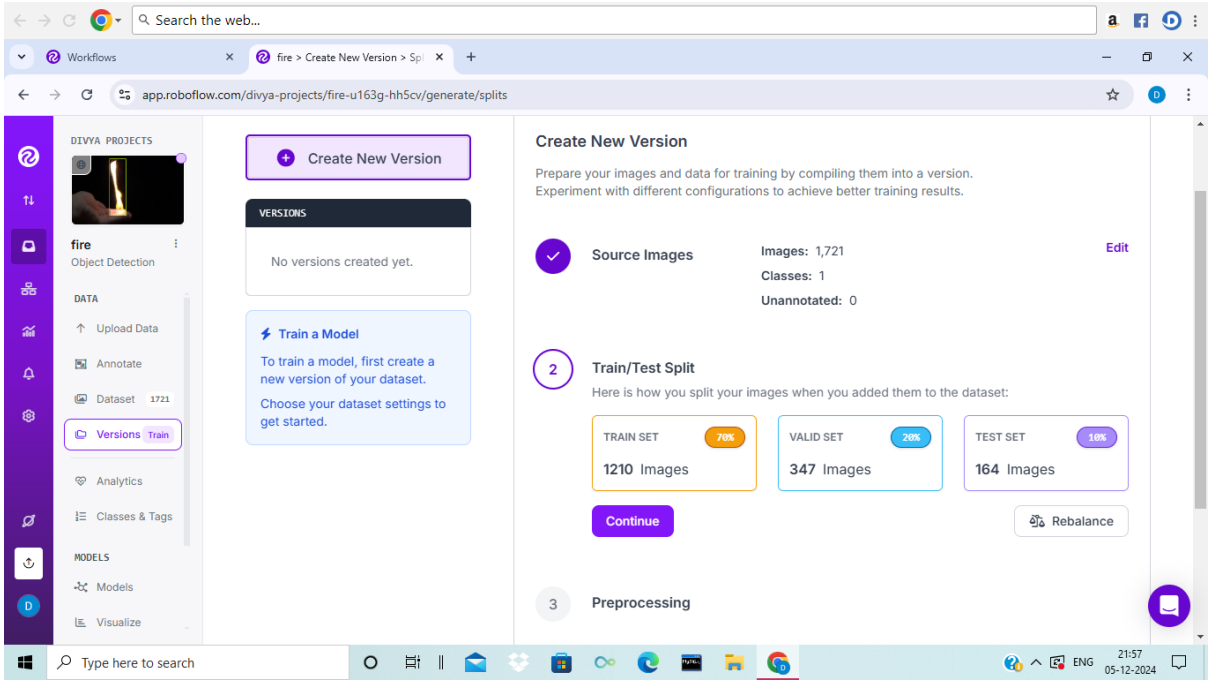


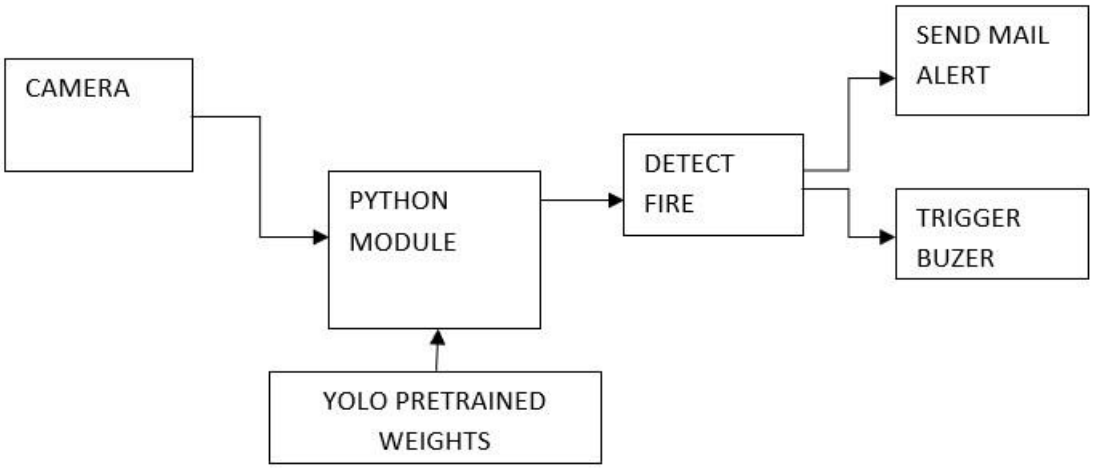
Figure 3.1: Dataset Snapshot


Search the web...
Workflows
fire > Create New Version > Sp
app.roboflow.com/divya-projects/fire-u163g-hh5cv/generate/splits
DIVYA PROJECTS
fire
Object Detection
DATA
Upload Data
Annotate
Dataset 1721
Versions Train
Analytics
Classes & Tags
MODELS
Models
Visualize
Create New Version
VERSIONS
No versions created yet.
Train a Model
To train a model, first create a new version of your dataset.
Choose your dataset settings to get started.
Create New Version
Prepare your images and data for training by compiling them into a version.
Experiment with different configurations to achieve better training results.
Source Images
Images: 1,721
Classes: 1
Unannotated: 0
Edit
2
Train/Test Split
Here is how you split your images when you added them to the dataset:
TRAIN SET 70%
1210 Images
VALID SET 20%
347 Images
TEST SET 10%
164 Images
Continue
Rebalance
3
Preprocessing
Type here to search
ENG
21:57
05-12-2024

CHAPTER 4
SYSTEM DESIGN

4.1 SYSTEM DESIGN

DESIGN OF THE PROPOSED SYSTEM :



The passive components of the system include data preprocessing, feature engineering, model selection scripts which were used to train and develop machine learning model. Source/input data which is in the form of videos is split into frames and preprocessed to convert it into a format that is suitable to be fed as input to pre-built models for feature extraction. The deep learning model returns a feature vector which is also known in transfer learning terminology as bottleneck features. In the next stage, the bottleneck features are passed through a classification model to obtain the result, which may be either fire of Non-fire. The classification model was built through training using the training data set.

The result of the classification is displayed to the user, and depending on the result, further actions are taken. If the result is a fire then, an email is sent to the concerned stakeholders along with the video frame and date-time stamp to alert them. The email to which the mail is delivered can be changed by the user.

YOLOV5 ARCHITECTURE:

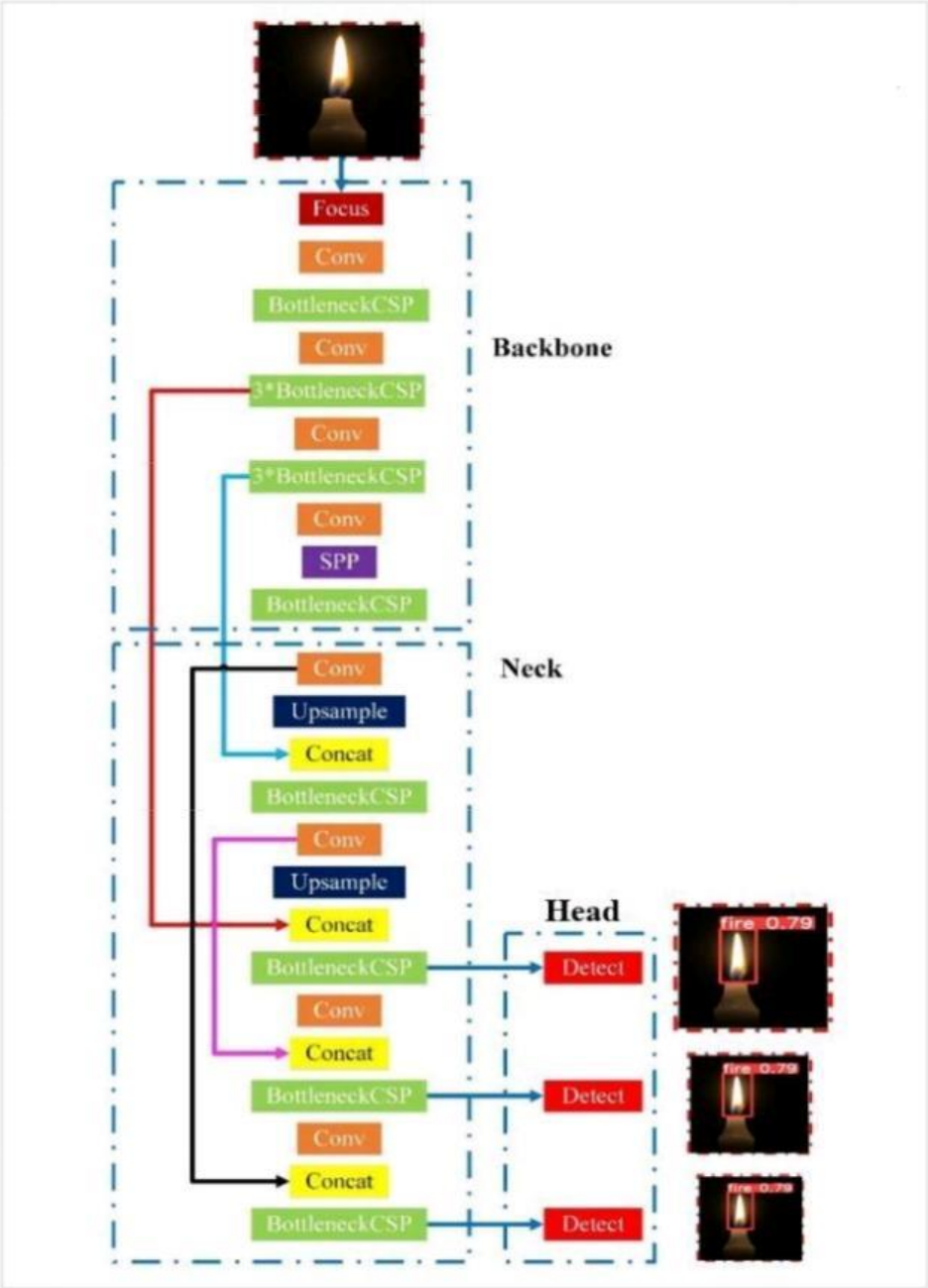
YOLOV5S NETWORK ARCHITECTURE YOLOv5 network [10,11] is the latest product of the YOLO architecture series. The detection accuracy of this network model is high, and the inference speed is fast, with the fastest detection speed being up of 140 frames per second. On the other hand, the size of the weight file of YOLOv5 target detection network model is small, which is nearly 90% smaller than YOLOv4**.**

ARCHITECTURE DESIGN ANALYSIS:

As YOLO v5 is a single-stage object detector, it has three important parts like any other single-stage object detector.

1. Model Backbone
2. Model Neck
3. Model Head

Fig no 4.3



The architecture of this project is designed for efficient and accurate real-time fire detection using YOLO. The process begins with image acquisition, where live video is captured via a camera. These raw images are then passed to the pre-processing module, which involves gray scaling and thresholding. These steps enhance image clarity by reducing noise and highlighting essential features. Next, the pre-processed images are fed into the feature extraction stage, where key patterns and details necessary for fire recognition are extracted. This extracted data is input to the YOLO model, a state-of-

the-art fire detection framework. The model leverages its backbone network for feature encoding and processes the data through its detection layers.

The final stage involves fire detection output, where the YOLO model identifies fire in the images, marking them with bounding boxes and labels. This modular design ensures the system handles diverse real-world scenarios, adapting to various lighting conditions. The architecture is structured for scalability and accuracy, emphasizing real-time performance. This pipeline is optimized to work on resource-constrained devices like surveillance camera, making it accessible and practical for end-users.

SYSTEM REQUIREMENTS:

Hardware Requirements

- Hard Disk : 20 GB or above
- RAM : 4 GB or above
- Processor Speed : 2.0 GHz and above
- Camera : System webcam/External standard camera

Software Requirements

- Operating System : Window 10
- Programing Language : Python
- Python Version : Python 3.7
- IDE : Visual Studio

CHAPTER 5

IMPLIMENTATION

5.1 IMPLIMENTATION

Implementation is the process of converting a new or a revised system design into an operational one. The objective is to put the new or revised system that has been tested into operation while holding costs, risks, and personal irritation to the minimum. A critical aspect of the implementation process is to ensure that there will be no disrupting the functioning of the organization. The best method for gaining control while implanting any new system would be to use well planned test for testing all new programs. Before production files are used to test live data, text files must be created on the old system, copied over to the new system, and used for the initial test of each program.

Another factor to be considered in the implementation phase is the acquisition of the hardware and software. Once the software is developed for the system and testing is carried out, it is then the process of making the newly designed system fully operational and consistent in performance.

Implementation is the most crucial stage in achieving a successful system and giving the user's confidence that the new system is workable and effective. Implementation of a modified application to replace an existing one. This type of conversation is relatively easy to handle, provide there are no major changes in the system.

PARALLEL CONVERSION TYPE OF IMPLEMENTATION

In this type of implementation both the current system and the proposed system run in parallel. This happens till the user gets the complete confidence on the proposed system and hence cuts of the current system.

PHASE - IN METHOD OF IMPLEMENTATION

In this type of implementation, the proposed system is introduced phase-by-phase. This reduces the risk of uncertainty of proposed system.

Each program is tested individually at the time of development using the data and has verified that this program linked together in the way specified in the programs specification, the computer system and its environment is tested to the satisfaction of the user. The system that has been developed is accepted and proved to be satisfactory for the user. And so the system is going to be implemented very soon. A simple operating procedure is included so that the user can understand the different functions clearly and quickly.

Initially as a first step the executable form of the application is to be created and loaded in the common server machine which is accessible to the entire user and the server is to be connected to a network. The final stage is to document the entire system which provides components and the operating procedures of the system.

Implementation is the stage of the project when the theoretical design is turned out into a working system. Thus it can be considered to be the most critical stage in achieving a successful new system and in giving the user, confidence that the new system will work and be effective.

The implementation stage involves careful planning, investigation of the existing system and constraints on implementation, designing of methods to achieve changeover and evaluation of changeover methods.

Implementation is the process of converting a new system design into operation. It is the phase that focuses on user training, site preparation and file conversion for installing a candidate system. The important factor that should be considered here is that the conversion should not disrupt the functioning of the organization.

5.2 CODE

```
import cv2
import numpy as np
import time
import pyttsx3
import sendmail
from pygame import mixer
import threading
engine = pyttsx3.init()
engine.setProperty("rate", 110)
mixer.init()

def buzzer():
    sound = mixer.Sound('alarm.wav')
    sound.play()


def voicebuzzer():
    engine.say('Fire detected')
    engine.runAndWait()




net = cv2.dnn.readNet("fire.weights","fire.cfg") #Tiny Yolo
classes = []
with open("fire.names","r") as f:
    classes = [line.strip() for line in f.readlines()]

layer_names = net.getLayerNames()
```

```
outputlayers = [layer_names[i - 1] for i in net.getUnconnectedOutLayers()]

#loading image
cap=cv2.VideoCapture(0) #0 for 1st webcam
font = cv2.FONT_HERSHEY_PLAIN
starting_time= time.time()
frame_id = 0
count_id = 0
obj_id = 0
while True:
    _,frame= cap.read()
    frame_id+=1

    height,width,channels = frame.shape
    #detecting objects
    blob = cv2.dnn.blobFromImage(frame,0.00392,(224,224),(0,0,0),True,crop=False)



    net.setInput(blob)
    outs = net.forward(outputlayers)
    #print(outs[1])



    class_ids=[]
    confidences=[]
    boxes=[]
    TrackedIDs = []
    for out in outs:
        for detection in out:
            scores = detection[5:]
```

```
        class_id = np.argmax(scores)
        confidence = scores[class_id]
        if confidence > 0.3:
            count_id += 1
            #object detected

            center_x= int(detection[0]*width)
            center_y= int(detection[1]*height)
            w = int(detection[2]*width)
            h = int(detection[3]*height)

            #get ID
            Id = int(obj_id)
            #rectangle co-ordinaters
            x=int(center_x - w/2)
            y=int(center_y - h/2)

            #cv2.rectangle(frame,(x,y),(x+w,y+h),(0,255,0),2)

            boxes.append([x,y,w,h])
            confidences.append(float(confidence))
            class_ids.append(class_id)

    indexes = cv2.dnn.NMSBoxes(boxes,confidences,0.4,0.6)

    for i in range(len(boxes)):
        if i in indexes:
            x,y,w,h = boxes[i]
            label = str(classes[class_ids[i]])
```

```
        confidence= confidences[i]
        color = (0,0,255)
        cv2.circle(frame,(center_x,center_y),1,(0,255,0),2)
        cv2.rectangle(frame,(x,y),(x+w,y+h),color,2)
        cv2.putText(frame,label+"
"+str(round(confidence,2)),(x,y+30),font,1,(255,255,255),2)
        if(label == 'fire' and count_id % 15 == 0):
            cv2.imwrite("final.jpg",frame)
            img_path = "final.jpg"

            threading.Thread(target=buzzer).start()
            threading.Thread(target=voicebuzzer).start()
            try:
                sendmail.sendalert()
            except:
                print("could not send mail")
            alert = 0

    cv2.putText(frame,"Monitoring",(10,50),font,2,(0,0,255),2)
    cv2.imshow("FIRE DETECTION",frame)
    key = cv2.waitKey(1)

    if key == 27: #esc key stops the process
        break

cap.release()
cv2.destroyAllWindows()
```

```
#----------------Send mail-------------

import smtplib
import ssl
from email.mime.text import MIMEText
from email.utils import formataddr
from email.mime.multipart import MIMEMultipart  # New line
from email.mime.base import MIMEBase  # New line
from email import encoders  # New line

sender_email = 'poojasreenair2001@gmail.com'
sender_name = 'pooja'
password = 'ogskeukjprzbvqzv'

receiver_emails = ['bdivyashree1@gmail.com']
receiver_names = ['divyashree']


filename = 'final.jpg'
def sendalert():
    for receiver_email, receiver_name in zip(receiver_emails, receiver_names):
        print("Sending email...")
        msg = MIMEMultipart()
        msg['To'] = formataddr((receiver_name, receiver_email))
        msg['From'] = formataddr((sender_name, sender_email))
        msg['Subject'] = 'FIRE DETECTED'

        try:
          with open(filename, "rb") as attachment:
                part = MIMEBase("application", "octet-stream")
                part.set_payload(attachment.read())
```

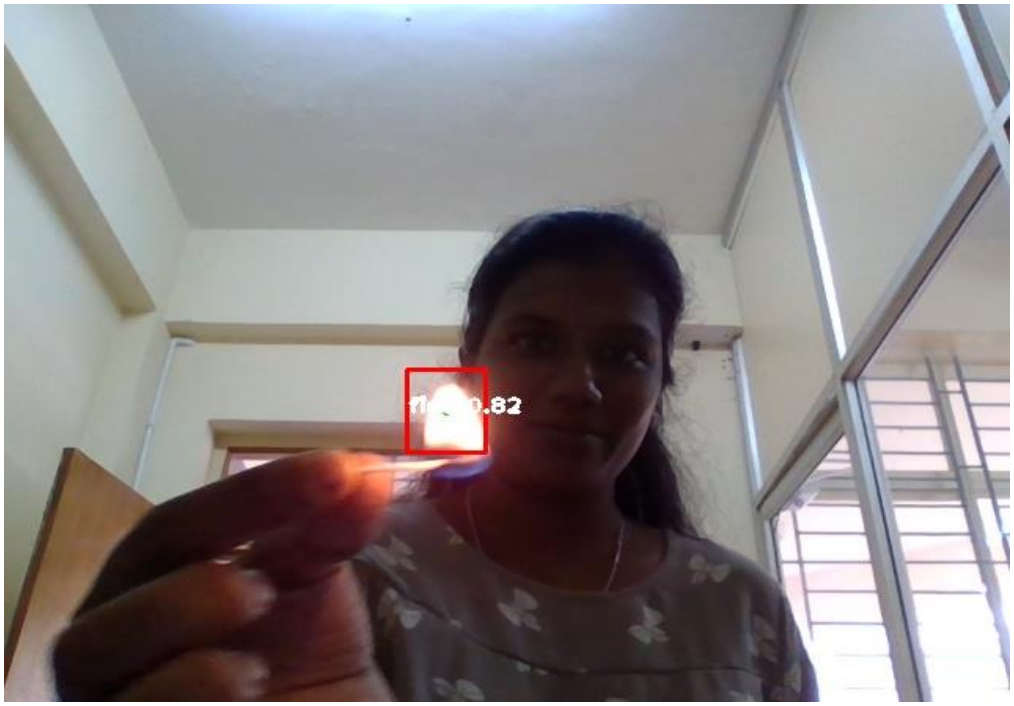
```
    encoders.encode_base64(part)

    part.add_header(
        "Content-Disposition",
        f"attachment; filename= {filename}",
    )
    msg.attach(part)
except Exception as e:
    print(f'Oh no! We didnt found the attachment!n{e}')
    break
try:
    # Creating a SMTP session | use 587 with TLS, 465 SSL and 25
    server = smtplib.SMTP('smtp.gmail.com', 587)
    context = ssl.create_default_context()
    server.starttls(context=context)
    server.login(sender_email, password)
    server.sendmail(sender_email, receiver_email, msg.as_string())
    print('Email sent!')
except Exception as e:
    print(f'Oh no! Something bad happened!n{e}')
    break
finally:
    print('Closing the server...')
    print('email sent')
    server.quit()
```

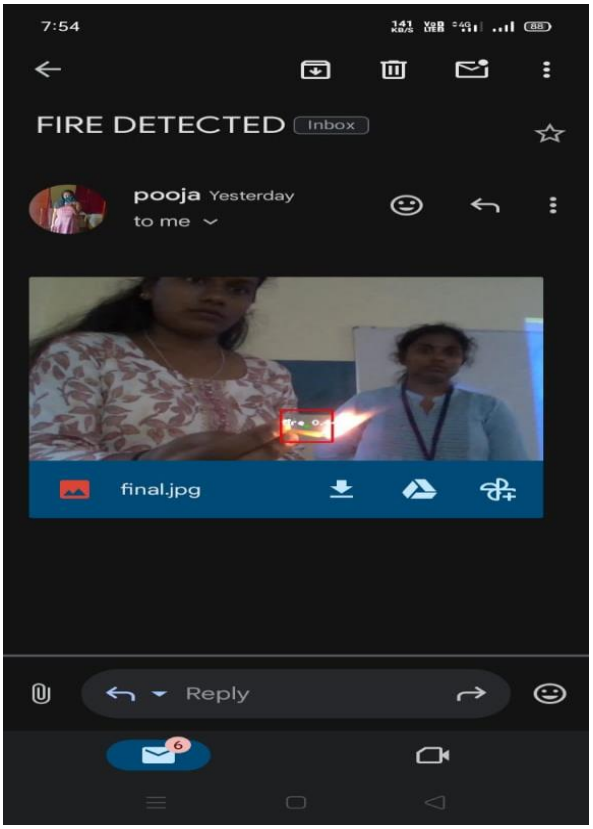
CHAPTER 6
RESULTS AND DISCUSSIONS

6.1 RESULTS AND DISCUSSIONS

The results obtained from this real-time furniture detection project demonstrate the effectiveness of the YOLOv5 model in accurately identifying fire within live video feeds. The system's performance was evaluated based on detection accuracy, speed, and the ability to process diverse lighting and environmental conditions. Using pre-processing techniques like gray scaling and thresholding, the feature extraction step enhanced the model's ability to distinguish object edges and textures effectively. YOLOv5 further leveraged its optimized architecture to detect furniture of varying sizes and orientations. Quantitative results showed high precision and recall rates, highlighting the robustness of the model. Real-time testing confirmed minimal latency, making the system suitable for dynamic environments. The discussion emphasizes how YOLOv5's modular structure enables efficient handling of feature-rich datasets, even in resource-constrained setups like a surveillance camera. An email alert feature has also been incorporated to our application to provide real time alerts to the concerned stakeholders. The project showcases the model's adaptability and speed in real-time applications, offering potential for future enhancements. The findings underline the system's practicality for real-world fire detection scenarios.

OUTPUT IMAGES





As soon as the fire is detected email will be sent to the given email address regarding fire detection with image in jpeg format.

CHAPTER 7
CONCLUSION

CONCLUSION

The present decade is marked by huge strides in areas of processing, computation and algorithms. This has enabled great progress in many fields including processing of surveillance video streams for recognizing abnormal or unusual events and actions. Fire accidents have caused death and destruction all over the world, consuming countless lives and causing billions in damages. This implies that developing an accurate, early, affordable fire-detection system is imperative Therefore, we have proposed a fire detection model for videos/video frames using transfer learning for deep learning. The models make use of ResNet-50, InceptionV3 and Inception-ResNet-V2 models to extract the features and various ML algorithms such as SVM, Logistic Regression, Naive Bayes and Decision Tree on the extracted features to detect fire in video frames. Looking at the results, ResNet-50 with SVM works best for our problem statement. Coming to the application on the whole, it works in real-time and has the ability to send alert emails along with offering a user-friendly graphical interface. It's cost-effective, reliable, robust, accurate compared to existing opto-electronic hardware and software-based systems in the market.

Future Enhancements:

The application can be enhanced by training the model with a larger dataset consisting of fires at various stages and dimensions. With higher GPU memory, we could use two deep learning models for feature extraction, whose output feature vectors are concatenated and classified to offer more robustness. An R-CNN model can be used to implement fire localization along with classification. We can also expect better deep learning architectures to emerge in the future, offering better feature extraction. The application will also offer a considerably better performance when run on machines having better processing power compared to existing one of which it has been developed.

CHAPTER 8
REFERENCES

8.1 REFERENCES

[1]. National Risk Survey Report - Pinkerton, FICCI (2018).

[2]. Janku P., Kominkova Oplatkova Z., Dulik T., Snopek P. and Liba J. 2018. “Fire Detection in Video Stream by Using Simple Artificial Neural”. Network. MENDEL. 24, 2 (Dec. 2018), 55–60.

[3]. Shen, D., Chen, X., Nguyen, M., & Yan, W. Q. (2018). “Flame detection using deep learning”. 2018 4th International Conference on Control, Automation and Robotics (ICCAR).

[4]. Li, C., & Bai, Y. (2018). “Fire Flame Image Detection Based on Transfer Learning”. 2018 5th IEEE International Conference on Cloud Computing and Intelligence Systems (CCIS).

[5]. K. Muhammad, J. Ahmad, I. Mehmood, S. Rho and S. W. Baik, “Convolutional Neural Networks Based Fire Detection in Surveillance Videos," in IEEE Access, vol. 6, pp. 18174-18183, 2018.

[6]. S. J. Pan and Q. Yang, "A Survey on Transfer Learning," in IEEE Transactions on Knowledge and Data Engineering, vol. 22, no. 10, pp. 1345-1359, Oct. 2010.

[7]. [Zhang, Qingjie & Xu, Jiaolong & Xu, Liang & Guo, Haifeng. (2016). “Deep Convolutional Neural Networks for Forest Fire Detection”.

[8]. K. Muhammad, J. Ahmad, Z. Lv, P. Bellavista, P. Yang and S. W. Baik, “Efficient Deep CNN-Based Fire Detection and Localization in Video Surveillance Applications,” in IEEE Transactions on Systems, Man, and Cybernetics: Systems, vol. 49, no. 7, pp. 1419-1434, July 2019.

[9]. Z. Jiao et al., “A Deep Learning Based Forest Fire Detection Approach Using UAV and YOLOv3,” 2019 1st International Conference on Industrial Artificial Intelligence (IAI), Shenyang, China, 2019.

[10]. Faming Gong ,1 Chuantao Li,1 Wenjuan Gong ,1 Xin Li,1 Xiangbing Yuan,2 Yuhui Ma and Tao Song. “A Real-Time Fire Detection Method from Video with Multi-feature Fusion”. Hindawi, Computational Intelligence and Neuroscience, Volume 2019.

[11]. L. Shao, F. Zhu and X. Li, “Transfer Learning for Visual Categorization: A Survey,” in IEEE Transactions on Neural Networks and Learning Systems, vol. 26, no. 5, pp. 1019-1034, May 2015.

[12]. S. Mohd Razmi, N. Saad and V. S. Asirvadam, “Vision-based flame detection: Motion detection & fire analysis,” 2010 IEEE Student Conference on Research and Development (SCOReD), Putrajaya, 2010, pp. 187-191.